

Desvios Condicionais e Operações Artiméticas



UNIFEI

Universidade Federal de Itajubá

IMC – Instituto de Matemática e Computação

Prof. Carlos Minoru Tamaki

Desvios condicionais

- Os processadores x86 possuem um grande conjunto de sinalizadores que representam o estado do processador, e as instruções de salto condicional podem ser ativadas em combinação.
 - **CF - carry flag**
 - Definido 1 para transporte ou empréstimo de bits; se 0 não ocorreu
 - **PF - parity flag**
 - Definido 1 se os bits de resultado contiverem um número par de bits “1”; 0 se contém número ímpar de “1”
 - **ZF - zero flags**
 - Definido 1 se o resultado for zero; 0 se o resultado for diferente de zero
 - **SF - sign flag**
 - Definido igual ao bit de resultado de ordem superior (0 se positivo, 1 se negativo)
 - **OF - overflow flag**
 - Definido se o resultado for um número positivo muito grande ou um número negativo muito pequeno (excluindo o bit de sinal) e não couber no operando de destino; caso contrário, é desmarcado.

Instruction	Description	signed-ness	Flags	short jump opcodes	near jump opcodes
JO	Jump if overflow		OF = 1	70	0F 80
JNO	Jump if not overflow		OF = 0	71	0F 81
JS	Jump if sign		SF = 1	78	0F 88
JNS	Jump if not sign		SF = 0	79	0F 89
JE JZ	Jump if equal Jump if zero		ZF = 1	74	0F 84
JNE JNZ	Jump if not equal Jump if not zero		ZF = 0	75	0F 85
JB JNAE JC	Jump if below Jump if not above or equal Jump if carry	unsigned	CF = 1	72	0F 82
JNB JAE JNC	Jump if not below Jump if above or equal Jump if not carry	unsigned	CF = 0	73	0F 83

JBE JNA	Jump if below or equal Jump if not above	unsigned	CF = 1 or ZF = 1	76	0F 86
JA JNBE	Jump if above Jump if not below or equal	unsigned	CF = 0 and ZF = 0	77	0F 87
JL JNGE	Jump if less Jump if not greater or equal	signed	SF <> OF	7C	0F 8C
JGE JNL	Jump if greater or equal Jump if not less	signed	SF = OF	7D	0F 8D
JLE JNG	Jump if less or equal Jump if not greater	signed	ZF = 1 or SF <> OF	7E	0F 8E
JG JNLE	Jump if greater Jump if not less or equal	signed	ZF = 0 and SF = OF	7F	0F 8F
JP JPE	Jump if parity Jump if parity even		PF = 1	7A	0F 8A
JNP JPO	Jump if not parity Jump if parity odd		PF = 0	7B	0F 8B
JCXZ JECXZ	Jump if %CX register is 0 Jump if %ECX register is 0		%CX = 0 %ECX = 0	E3	

JRCXZ - jump if %RCX register is 0 - %RCX = 0 – opcode E3

Instruções Aritméticas

- Instrução **INC**

- A instrução *inc* é usada para incrementar um operando em um. Ele funciona com um único operando que pode ser um registrador ou um endereço de memória.

- Syntaxe:

inc operando

O operando pode ser de 8-bit, 16-bit, 32-bit ou 64-bit.

- Exemplo:

`inc ebx` ; incrementa o registrador ebx de 32-bit
`inc dl` ; incrementa o registrador dl de 8-bit
`Inc DWORD [count]` ; incrementa a variável count

Instrução **DEC**

A instrução **dec** é usada para decrementar um operando em um. Ele funciona com um único operando que pode ser um registrador ou um endereço de memória.

- Syntax:

dec operando

- O operando pode ser de 8-bit, 16-bit, 32-bit ou 64-bit.

- Exemplo:

dec ebx ; decrementa o registrador ebx de 32-bit
dec dl ; decrementa o registrador dl de 8-bit
dec DWORD [count] ; decrementa a variável count

Instruções **ADD** e **SUB**

- As instruções *add* e *sub* são usadas para realizar simples operações aritméticas binárias de adição/subtração dados do tamanho em byte, word, doubleword ou quadword. Por exemplo, para adicionar ou subtrair 8-bit, 16-bit, 32-bit ou operandos de 64-bit, respectivamente.
- Syntax:
 add/sub destino, origem
- Estas duas instruções podem ser usadas em operações do tipo:
 - registrador para registrador
 - memória para registrador
 - registrador para memória
 - registradores para dados constantes
 - memória para dados constantes
- Contudo, como em outras instruções operações de memória-para-memória não são possíveis usando o estas instruções. A operação do add ou sub podem alterar as flags de overflow ou de carry.

As instruções **MUL/IMUL**

- Existem duas instruções para operações de multiplicações com binários. A instrução *MUL* (Multiply) trata de dados sem sinal e a instrução *IMUL* (Integer Multiply) trata de dados com sinal. Ambas as instruções afetam o valores das flags Carry e Overflow.
- Syntax:
mul/imul multiplicador
- O multiplicando em ambos os casos vai estar num acumulador, dependendo do tamanho do multiplicando e do multiplicador e do produto gerado, dependendo do tamanho dos operandos, serão armazenados em dois registradores. A seguir é apresentada uma tabela com os quatro diferentes casos possíveis usando a instrução *mul*:

[i]mul r/m

Operando 1	Operando 2	Destino
AL	reg/mem8	AX
AX	reg/mem16	DX:AX
EAX	reg/mem32	EDX:EAX
RAX	reg/mem64	RDX:RAX

Caso	Cenário
1	Quando dois bytes são multiplicados: Se o multiplicando está no registrador <i>al</i>, e o multiplicador é um byte na memória ou em outros registrador. O produto será armazenado em <i>ax</i>. Os 8 bits mais significativos do produto são armazenados em <i>ah</i> e os 8 menos significativos em <i>al</i>.
2	Quando duas words são multiplicadas: O multiplicando é armazenado no registrador <i>ax</i>, e o multiplicador é uma word em memória ou um valor em um outro registrador. Por exemplo, ao usar a instrução <i>mul dx</i>, você deve armazenar o multiplicador em <i>dx</i> e o multiplicando em <i>ax</i>. O resultado produzido é uma doubleword, e que por isso irá precisar de dois registradores. Os bytes mais significativos serão armazenados no registrador <i>dx</i> e os bytes menos significativos no registrador <i>ax</i>.

Caso	Cenário
3	Quando duas doublewords são multiplicadas: Quando duas doublewords são multiplicadas, o multiplicando deverá estar armazenado em <i>eax</i> e o multiplicador deverá estar armazenado em memória ou em outro registrador. O produto gerado será armazenado nos registradores <i>edx:eax</i>.
4	Quando duas quadwords são multiplicadas: Quando duas quadwords são multiplicadas, o multiplicando deverá estar armazenado em <i>rax</i> e o multiplicador deverá estar armazenado em memória ou em outro registrador. O produto gerado será armazenado nos registradores <i>rdx:rax</i>.

– Exemplo:

```
mov al, 10
```

```
mov dl, 25
```

```
mul dl
```

```
...
```

```
mov dl, 0ffh ; dl = -1
```

```
mov al, 0beh ; al = -66
```

```
imul dl
```

As instruções **DIV/IDIV**

- As operações de divisão geram dois elementos, o quociente e o resto. No caso na multiplicação, o overflow não acontece porque são usados dois registradores de tamanho duplo para manter o produto. Contudo, no caso da divisão, o overflow pode acontecer. O processador gera uma interrupção quando ele ocorre.
- A instrução `div` (Divide) é usada para dados sem sinal e a instrução `idiv` (Integer Divide) é usada para dados com sinal.
 - Syntax:
div/idiv divisor
 - O dividendo é armazenado no acumulador. Ambas as instruções podem trabalhar com operandos de 8-bit, 16-bit, 32-bit e 64-bit. A operação afeta todas as seis flags. A próxima secção explica os quatro casos da divisão com operadores de diferentes tamanhos:

[i]div r/m

Operando 1	Operando 2	Quociente	Resto
AX	reg/mem8	AL	AH
DX:AX	reg/mem16	AX	DX
EDX:EAX	reg/mem32	EAX	EDX
RDX:RAX	reg/mem64	RAX	RDX

Caso	Cenário
1	Quando o divisor é 1 byte: É assumido que o dividendo se encontra no registrador <i>ax</i> (16-bits). Depois da divisão, o quociente encontra-se no registrador <i>al</i> e o resto vai para o registrador <i>ah</i> .
2	Quando o divisor é 1 word: É assumido que o dividendo tem 32 bits de comprimento e encontra-se nos registradores <i>dx:ax</i> . Os 16 bits mais significativos encontram-se <i>dx</i> e os menos significativos encontram-se em <i>ax</i> . Depois da divisão, os 16-bit do quociente vão para o registrador <i>ax</i> e os 16-bit do resto vão para <i>dx</i> .

Caso	Cenário
3	Quando o divisor é uma doubleword: Assume-se que o dividendo tem 64 bits de comprimento e encontra-se em <i>edx:eax</i>. Os 32 bits mais significativos encontram-se em <i>edx</i> e os 32 bits menos significativos encontram-se em <i>eax</i>. Após a divisão, os 32-bit do quociente vão para o registrador <i>eax</i> e o 32-bit do resto vão para o registrador <i>edx</i>.
4	Quando o divisor é uma quadword: Assume-se que o dividendo tem 128 bits de comprimento e encontra-se em <i>rdx:rax</i>. Os 64 bits mais significativos encontram-se em <i>rdx</i> e os 64 bits menos significativos encontram-se em <i>rax</i>. Após a divisão, os 64-bit do quociente vão para o registrador <i>rax</i> e o 64-bit do resto vão para o registrador <i>rdx</i>.

Bibliografia

- Vários sites da internet